

AI VIDEO PIPELINE

Architecture Overview

How the automation is built, how steps hand off to each other,
how files move through the system, and how concurrent requests are handled.

Stack & State

Step Sequencing

File Handoff

Concurrency



Thank you for the thoughtful questions

Questions like these are genuinely useful — they help sharpen the system on both sides, and it's great to have this kind of exchange. The next few pages walk through the details behind each one.

What the bot is built with

It's custom Python — not n8n, not a no-code automation tool, and not Slack's Workflow Builder. Slack Bolt handles the real-time connection; everything after that is our own code.



Custom Python

Hand-written modules per pipeline (dave_runner.py, jiny_runner.py, blitz_runner.py) plus shared backends for each AI service.



Slack Bolt (Socket Mode)

One persistent WebSocket connection listens for messages in the channel and routes them to the right pipeline handler.



In-memory state object

A single "pipeline" object holds what step we're on, which brief, which character — kept alive for the duration of one request.



How one request is remembered start-to-finish

When someone triggers a pipeline, the bot resets a shared state object and writes the brief, the chosen character, the platform, and a "waiting_for" marker onto it. Every subsequent message in that channel is checked against that marker to decide what it means — a scene count, a menu choice, or free text.

Why it's built this way



A quick naming note: "Jiny" is the same thing referred to earlier as the "**Motion pipeline.**" Each pipeline was given a character-like name so it reads naturally in Slack chat — "Dave," "Jiny," and "Blitz" are all names for that same idea.



Built around subscriptions we already pay for

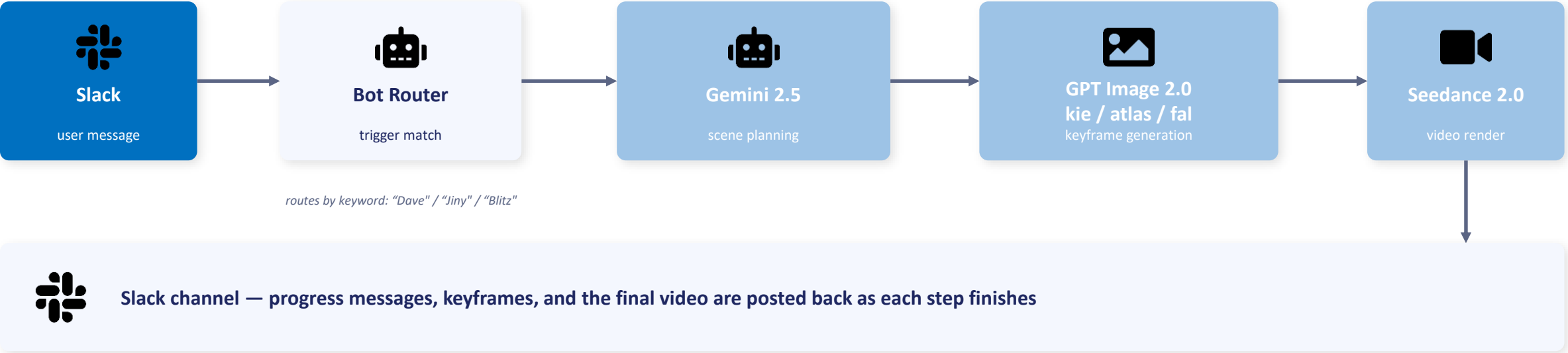
The whole design is intentionally built to make use of AI platform accounts we're already subscribed to, for cost reasons. It's not the fastest possible setup — it's the most cost-effective one available today. A faster, simpler pipeline is planned as a future upgrade once it makes sense.



Built so the computer keeps working, unattended

The original goal was simple: the computer should keep producing videos even when nobody is sitting at the desk. That's why every step — planning, images, video — runs on its own and reports progress back to Slack, instead of needing someone to babysit each click.

End-to-end architecture



Three parallel pipelines share this shape: Dave (fully automatic), Jiny (step-by-step approval), Blitz (fast, API-only track with a choice of backend per run).

Dave No human checkpoints — plans, generates, and posts automatically.	Jiny Pauses for approval after each step (STEP 1–4).	Blitz Character + backend chosen up front, then runs straight through.
--	--	--

How one step knows the previous one is done

Each step is a normal Python function call, not an event listener — so "done" simply means the function returned successfully. There's no polling for a signal between steps; the code runs Gemini, waits for that call to return, then runs GPT Image, waits again, then runs the video model.

Dave — fully automatic

- 1 STEP 1 Gemini writes the scene brief
- 2 STEP 2 GPT Image renders each keyframe
- 3 STEP 3 Runway/Seedance renders video
- 4 No approval needed — but every step posts to Slack as it happens

Jiny — approval between steps

- 1 STEP done → `pipeline.waiting_for = "step_approval"`
- 2 Bot posts a numbered menu, then stops
- 3 Next message is read as a menu choice, not free text
- 4 Choice clears `waiting_for` → next step launches

The "waiting_for" field is the key mechanism: it's the only thing that decides whether an incoming Slack message is a scene count, an approval, or a fresh request.



So how do you actually check on Dave? Even with no approval step, every stage — scene plan, each keyframe, the final video — is posted to Slack the moment it's ready, so you can always see what's happening without waiting for the whole run to finish.

Balancing render quality and turnaround time

Seedance produces the best quality, but it's also the slowest to render. When something needs to go out fast, the pipeline can pull from Kling or Gemini credits — trading a bit of quality for speed. Seedance can also be run on paid credits directly, which keeps the same quality and speed but costs more per render.

Seedance

current Runway Unlimited plan

Best quality

Highest visual fidelity of the three. Used by default for final output when there's time to let it finish.

Trade-off: noticeably slower to render than the alternatives.

Kling, Gemini credits

Fast alternative

Swapped in mid-project when a deadline is close and Seedance would take too long to finish in time — a lever to pull when timing matters.

Trade-off: quality is a step below Seedance.

Seedance credits

Fast, higher credit cost

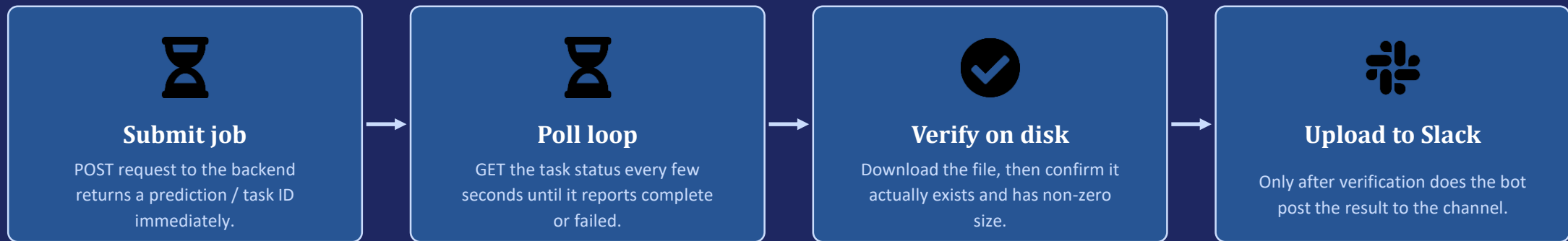
Same Seedance quality, rendered by spending API credits directly instead of the Unlimited plan — this route is fast, but each render draws down paid credits.

Trade-off: costs more per render than the other two options.

This isn't an either/or choice made once — the platform can be switched mid-project depending on how urgent that particular request is.

The step that takes a bit longer: video rendering

Scene planning and keyframe images each finish in seconds. Video rendering (Seedance / Runway) is the one step that can take minutes — so the bot blocks on it deliberately, inside a background thread, and treats "submit → poll → verify" as one unit of work per shot.



Retry policy

- ✓ Before retrying, the code checks whether the previous attempt actually finished first — avoids wasted API credits and duplicate renders.
- ✓ Transient errors (HTTP 503 / 429 / 500) are retried automatically with exponential backoff (e.g. Gemini scene planning: up to 3 attempts).
- ✓ Time spent in a rate-limited / throttled state doesn't count against the overall timeout — only active processing time does.

Where files live, and how they hand off

Everything is local disk, organized by role. Each step reads what the previous step wrote and passes a plain file path forward — no database, no object storage.



Reference images

```
characters/{name}/identity/
```

One master portrait per character (e.g. Dave_ID_master.png). Read fresh every run — never copied elsewhere.



Keyframes

```
assets/keyframes/
```

Output of the image step. The very next step (video) reads this same path directly as its input image.



Final video

```
generations/final/
```

Verified (exists + non-zero size) before upload.
Draft files stay until a run is explicitly confirmed.

Handoff is always a Path passed directly between function calls in the same process — there's no queue, message bus, or shared cache to keep in sync.



Both Dave and Jiny work the same way end to end: every step from storyboard to final video is visible in Slack as it happens, and the finished files land in a fixed folder on the desktop.

What happens when two people ask at once

Today, two things can already run at the same time: the Dave + Jiny group, and Blitz. This works because Dave and Jiny share the same structure and the same underlying platform, while Blitz runs on a separate platform entirely — so the two groups never compete for the same resource.



Dave + Jiny group

Same structure, same platform. Because Blitz runs on a separate platform, this group and Blitz don't compete for the same resource and can run side by side. (Concurrency *within* this group — for example, calling Dave and Jiny at the same time — is still being separately verified.)



Blitz

Runs on a different platform from Dave/Jiny, by design. That separation is exactly what lets it run at the same time as the other group, without either one waiting on the other.



Still being actively developed

This concurrency work is one of the areas being actively pushed forward right now. The end goal: every person should be able to call their own character at the same time as everyone else, with zero collisions between them — not just two groups running side by side, but full concurrency across every user.

SUMMARY

Four questions, in short

Built with?

Custom Python + Slack Bolt (Socket Mode) — no n8n, no Workflow Builder.

Step order?

Sequential function calls; Dave has no approval step but posts every stage to Slack live.

Files?

Local disk, role-based folders (identity / keyframes / final); paths passed directly, no queue.

Concurrency?

Dave+Jiny and Blitz already run side by side today; full multi-user concurrency is the active next goal.



Happy to put together a fuller architecture diagram or a written setup note if that's useful for your side — just say which parts you'd like more detail on.